Trabalho de Conclusão de Curso de Engenharia da Computação

RADARTORRES:
PROTÓTIPO PARA DETECÇÃO, LOCALIZAÇÃO E SELEÇÃO AUTOMÁTICA DE TORRE COM CONTROLADOR HEURÍSTICO-NEUROEVOLUTIVO

RadarTorres:
a prototype for target detection, localization and automatic tower selection with a heuristic-neuroevolutionary controller

Gabriel Vasconcellos de Morais[1]
Matheus Emanoel Gomes Souza[2]
Anker Douglas Loss[3]

RESUMO

A construção de protótipos de sensoriamento com hardware acessível é um recurso didático relevante em Engenharia da Computação, por permitir explorar de forma prática conceitos de aquisição de dados, rastreamento e tomada de decisão automática. Este trabalho apresenta o RadarTorres, um protótipo que detecta um alvo em movimento, estima sua posição e decide, entre um conjunto de torres candidatas posicionadas ao redor de uma base central, qual delas deve responder a esse alvo – problema central que o trabalho busca resolver: como identificar, localizar e atribuir automaticamente a torre mais adequada a cada alvo, de forma auditável e sem depender de sensores ou atuadores de uso industrial. O sistema combina dois sensores ultrassônicos (que medem a distância pelo eco do ultrassom) com quatro câmeras de 720p, processa quadros de vídeo (*frames*) e classifica o alvo por sua cor, formato e tamanho. O protótipo converte as leituras de ângulo e distância em posição cartesiana, exibindo-as em tempo real em um radar circular, seleciona automaticamente a torre mais adequada para cada alvo e registra em auditoria toda ação executada. Essa seleção combina uma heurística determinística, escrita como regras explícitas de engenharia, com uma rede neural que aprende, por meio de um algoritmo genético próprio, a refinar essa escolha e o instante do acionamento. O treinamento dessa rede ocorre inteiramente antes da operação do sistema, em um ambiente simulado que executa gerações de episódios de teste até evoluir os parâmetros da rede; concluído o treinamento, esses parâmetros são incorporados de forma fixa à aplicação, de modo que, durante a execução real, o sistema apenas aplica o que foi aprendido previamente, sem ajustar seus parâmetros em tempo real. O processamento e a tomada de decisão são concentrados em uma aplicação desktop, com arquitetura pensada para permitir evolução

[1]Graduando(a) em Engenharia da Computação pela Faculdade do Centro Leste – UCL. E-*mail*: gabrielmorais@ucl.br
[2]Graduando(a) em Engenharia da Computação pela Faculdade do Centro Leste – UCL. E-*mail*: matheusemanoel@ucl.br
[3]Orientador(a). Mestre em xxx pela xxx. E-*mail*: anker@ucl.br

futura sem exigir hardware adicional. Discutem-se, por fim, as limitações inerentes à abordagem de hardware acessível e de baixo custo adotada, e o fato de a validação do protótipo, até o momento, ter sido realizada inteiramente por simulação de software, já que a estrutura física ainda não foi montada nem testada.

Palavras-chave: Detecção e rastreamento de alvos; Seleção automática de torre; Controlador heurístico-neuroevolutivo; Arduino; Protótipo.

ABSTRACT

This work presents RadarTorres, a prototype developed as a Final Course Project in Computer Engineering that detects a moving target, estimates its position, and decides, among a set of candidate towers placed around a central base, which one should respond to that target – the central problem this work addresses: how to automatically identify, localize, and assign the most suitable tower to each target, in an auditable way and without relying on industrial-grade sensors or actuators. The system combines two ultrasonic sensors (which measure distance by ultrasound echo) with four 720p cameras, processes video frames, and classifies the target by color, shape, and size. The prototype converts angle and distance readings into Cartesian position, displaying them in real time on a circular radar, automatically selects the most suitable tower for each target, and logs every executed action for auditing. This selection combines a deterministic heuristic, written as explicit engineering rules, with a neural network that learns, through a custom genetic algorithm, to refine that choice and the activation timing. Training of this network happens entirely before the system is put into operation, in a simulated environment that runs generations of test episodes until the network's parameters evolve; once training is complete, these parameters are embedded as fixed values into the application, so that, during actual execution, the system only applies what was previously learned, without adjusting its parameters in real time. Processing and decision-making are concentrated in a desktop application, with an architecture designed to allow future evolution without requiring additional hardware. Finally, the limitations inherent to the accessible, low-cost hardware approach adopted are discussed, as is the fact that validation of the prototype has so far been carried out entirely through software simulation, since the physical structure has not yet been assembled or tested.

Keywords: Target detection and tracking; Automatic tower selection; Heuristic-neuroevolutionary controller; Arduino; Prototype.

1 Introdução

Sistemas automáticos de detecção e resposta a alvos exigem, além de identificar um estímulo, rastrear com precisão sua posição e aplicar regras de decisão que evitem respostas indevidas – tarefas comumente resolvidas com sensores de radar, de custo elevado. Como alternativa a esse tipo de equipamento, este trabalho adota técnicas de visão computacional, aliadas a hardware acessível, como parte central da arquitetura do protótipo. Protótipos didáticos construídos dessa forma desempenham papel importante nesse processo, por permitirem que esses conceitos sejam explorados de forma prática e replicável em laboratório, reproduzindo os princípios de sensoriamento, rastreamento e decisão presentes em sistemas mais sofisticados.

O protótipo resultante, denominado RadarTorres, combina sensores ultrassônicos e câmeras para detectar um alvo, estimar sua posição em um radar circular e decidir, entre um conjunto de torres candidatas dispostas ao redor de uma base central, qual delas deve responder a esse alvo. Essa decisão não é tomada apenas por regras fixas: uma rede neural, evoluída por um algoritmo genético próprio, aprende a refinar a escolha da torre e o instante do acionamento, sempre dentro dos limites que uma camada de regras explícitas (a heurística) já considera seguros. Esse aprendizado acontece em duas etapas bem separadas: antes da operação do sistema, em um ambiente de treinamento simulado, onde a rede evolui ao longo de muitas gerações de episódios de teste; e, depois de treinada, apenas na forma de inferência durante a execução real, sem qualquer ajuste adicional de seus parâmetros em tempo de operação. O problema central que este trabalho busca resolver é, portanto, como automatizar essa cadeia – detectar, localizar e selecionar a torre mais adequada a cada alvo – de forma auditável e replicável em laboratório, sem depender de sensores ou atuadores de uso industrial.

2 Desenvolvimento

2.1 Revisão da literatura e fundamentação teórica

Sistemas de detecção e rastreamento de alvos costumam recorrer a sensores como radar, de custo elevado. Como alternativa, sensores acessíveis podem ser combinados de modo que as limitações de um sejam compensadas pelas qualidades do outro. Sensores ultrassônicos, como o HC-SR04, medem distância com boa confiabilidade em curto alcance, mas não distinguem o tipo do obstáculo; câmeras, por sua vez, fornecem informação semântica, porém dependem das condições de iluminação e exigem maior poder de processamento (MAJONI et al., 2025). A fusão de sensores (*sensor fusion*) combina as duas modalidades para aproveitar essa complementaridade.

Majoni et al. (2025) descrevem uma arquitetura em que um Arduino Mega trata o sensoriamento ultrassônico de baixa latência, enquanto um Raspberry Pi processa as imagens da câmera com uma rede YOLOv4-tiny, com comunicação serial entre os dois. Essa divisão de papéis é análoga à do RadarTorres, em que o Arduino realiza a aquisição e a aplicação desktop concentra o processamento.

FALTOU DETALHE: Conferir na fonte original (Majoni et al., 2025) se a divisão "Arduino Mega trata ultrassom + Raspberry Pi roda YOLOv4-tiny + comunicação serial" está descrita assim no paper, antes de manter a analogia como está.

A literatura também mostra que sensores ultrassônicos acessíveis podem complementar outras fontes de sensoriamento em sistemas de rastreamento, com a precisão melhorada por predição baseada em filtro de Kalman e fusão com a distância medida pelos ultrassônicos (BAEK et al., 2020). O mesmo trabalho observa que as leituras ultrassônicas se tornam ruidosas em distâncias maiores, o que motiva o uso de filtragem.

Definidos o alvo e sua posição, resta decidir qual recurso deve responder. Na literatura de defesa, esse problema é tratado como avaliação de ameaças e atribuição de armas (*Threat Evaluation and Weapon Assignment*, TEWA), no qual as informações dos sensores alimentam as etapas de detecção, rastreamento e identificação, seguidas da avaliação da ameaça e da atribuição dos recursos. As abordagens desse tipo de problema costumam ser classificadas em ameaça por ameaça, em que os recursos são atribuídos sequencialmente, e multiameaça, em que o conjunto de ameaças é considerado simultaneamente.

O RadarTorres adota uma abordagem simples, por proximidade, aplicada alvo a alvo, com propósito exclusivamente demonstrativo (laser de baixa potência, LED ou simulação, sem armamento real); a abordagem multiameaça fica como trabalho futuro.

O refinamento dessa escolha por proximidade – e o instante de disparo – é feito por uma pequena rede neural *feedforward* (6 entradas, 1 camada oculta de 8 neurônios, 2 saídas, ativação tanh), cujos pesos não são ajustados por retropropagação, mas evoluídos por um algoritmo genético próprio (seleção por torneio, *crossover* uniforme, mutação gaussiana, elitismo), auditável e sem dependência de biblioteca de aprendizado de máquina. Such et al. (2017) mostram que algoritmos genéticos são uma alternativa competitiva ao treino por gradiente em tarefas de aprendizado por reforço, em redes de porte bem maior do que a aqui empregada (até quatro milhões de parâmetros, contra os 74 parâmetros da rede do RadarTorres). Esse resultado sustenta a escolha do algoritmo genético como método de treinamento, mas não implica, por si só, que aumentar o número de camadas ou neurônios seja desnecessário; por isso, a estratégia adotada para ampliar o desempenho do controlador prioriza o ajuste dos hiperparâmetros da busca evolutiva (tamanho de população, paralelização da avaliação de fitness, enriquecimento do sinal de recompensa), sem descartar, a priori, uma rede maior caso evidências futuras mostrem que a capacidade atual seja o fator limitante.

FALTOU DETALHE: Faltam referências sobre: restrição de ações válidas (action masking) em RL, "shielding" em RL seguro (camada de segurança vetando ações de uma política aprendida) e neuroevolução aplicada a atribuição de armas/TEWA especificamente – fundamentariam a arquitetura híbrida heurística+rede melhor do que só a referência genérica de neuroevolução hoje citada.

O impacto do treinamento virtual sobre a execução física do controlador é atenuado por uma decisão de projeto central: o mesmo controlador híbrido – heurística e rede neural combinadas – que decide no episódio simulado é o que decide em produção, consultado pelo algoritmo responsável por selecionar a torre a partir das leituras do Arduino real – não há conversão para um formato intermediário (ONNX, TensorFlow Lite) nem reimplementação em outra linguagem ou plataforma, fontes comuns de divergência entre o comportamento treinado e o executado em sistemas de aprendizado por reforço aplicados

a robótica. O genoma evoluído é um vetor plano de pesos e vieses, embutido diretamente na aplicação por uma etapa automática de build, que grava o melhor genoma treinado no executável, de onde é carregado por um componente dedicado em tempo de execução; o que muda do ambiente de treino para a operação real é apenas a origem dos dados de entrada (leitura simulada ou leitura do Arduino), nunca a função de decisão em si. Os parâmetros que definem o episódio simulado – alcance de engajamento e certeza mínima para autorizar disparo – também deixaram de ser constantes fixas no código e passaram a ser lidos das mesmas configurações de alcance de mira e de limiar de certeza usadas pelo sistema real, de modo que recalibrar o alcance da câmera ou o limiar de certeza no painel de Configurações recalibra também o mundo de treino, reduzindo a divergência entre o que foi aprendido e o que será executado. Por fim, a rede neural nunca decide isoladamente: ela apenas pondera a escolha dentro do que a camada heurística determinística já considera válida (nunca uma torre ocupada, nunca fora de alcance), de modo que a arquitetura foi concebida para limitar o pior caso de uma generalização imperfeita do treino virtual para a física – no limite, o controlador regrediria ao comportamento puramente heurístico, nunca a uma decisão arbitrária; trata-se, porém, de uma propriedade de projeto ainda não submetida a validação em hardware físico real.

2.2 Procedimentos metodológicos

O percurso metodológico deste trabalho não seguiu o fluxo tradicional de especificação prévia seguida de implementação: o protótipo RadarTorres já existia, em desenvolvimento incremental, antes do levantamento formal de requisitos. Optou-se, portanto, por catalogar os requisitos funcionais e não funcionais diretamente a partir do que já estava implementado no código-fonte da aplicação desktop e da documentação técnica já produzida ao longo do projeto, em vez de defini-los a priori em um documento de especificação anterior à codificação. O treinamento do controlador por neuroevolução, descrito na Seção 2.1, segue essa mesma lógica incremental: a arquitetura híbrida heurística+rede neural foi definida e ajustada em paralelo ao restante do sistema, não a partir de um plano de treinamento fechado desde o início.

Esse levantamento resultou em 25 requisitos funcionais (RF01–RF25) e 20 requisitos não funcionais (RNF01–RNF20), classificados segundo dois critérios objetivos: a **prioridade**, atribuída conforme a criticidade de cada requisito para a operação central do sistema (comunicação serial, rastreamento, seleção de torre, modos de operação e segurança recebem prioridade Alta; suporte administrativo e ferramental, Média; conveniências de interface, Baixa); e o **status**, que reflete o estado real de implementação constatado no código no momento do levantamento – Implementado, Parcial ou Planejado –, e não a intenção original de projeto. Cada requisito foi, além disso, rastreado até a classe ou método do código-fonte que o realiza, registro mantido em uma matriz de rastreabilidade dedicada, de modo que qualquer afirmação sobre o grau de implementação do protótipo possa ser verificada diretamente no código. Divergências encontradas entre o comportamento observado no código e a nomenclatura ou fluxo originalmente esperado também foram registradas explicitamente, em vez de omitidas ou corrigidas silenciosamente na documentação.

A parte física do RadarTorres (torres, motores, sensores montados em campo) ainda não foi implementada nem validada. Toda a validação funcional relatada neste

trabalho foi realizada por simulação de software: a lógica de rastreamento de alvos e de seleção automática de torre foi exercitada de ponta a ponta sem depender de uma porta serial conectada a hardware real, o que impõe um limite claro ao escopo dos resultados relatados – eles atestam a corretude da lógica de decisão implementada em software, mas não substituem uma validação de campo com sensores e torres físicas, deixada como trabalho futuro.

A mesma ressalva se aplica, com reforço adicional, ao controlador treinado por neuroevolução: o mundo simulado em que os genomas são avaliados é uma aproximação idealizada do ambiente físico – os alvos nascem na borda de um círculo e avançam de forma quase radial, sem o ruído típico de leituras ultrassônicas reais; a certeza de identificação do alvo pela câmera é modelada como uma função sintética que cresce com o tempo de observação, e a probabilidade de errar o disparo é uma função assumida, crescente com a velocidade do alvo – nenhuma delas calibrada com dados de campo, já que o hardware ainda não foi montado. Essa distância entre simulação e realidade física não é apenas teórica: durante este trabalho, identificou-se e corrigiu-se uma falha em que a certeza de identificação do alvo era sorteada uma única vez, no nascimento, e nunca era atualizada – cerca de 70% dos alvos simulados nasciam permanentemente abaixo do limiar mínimo de disparo e jamais podiam ser abatidos, independentemente da qualidade do genoma, mascarando qualquer evolução real do controlador sob a aparência de estagnação. O episódio é citado aqui como evidência concreta de que uma premissa equivocada na simulação pode invalidar silenciosamente o processo de treinamento sem que isso seja percebido pelas métricas agregadas, reforçando a necessidade de validação cruzada entre o comportamento do genoma treinado e o comportamento observado em campo, com hardware físico, antes de se confiar no controlador em um cenário de operação real – validação que permanece como trabalho futuro, nos mesmos termos já apontados para o restante do protótipo.

FICOU BOM: Esse episódio (bug da certeza estática mascarando estagnação evolutiva) é a contribuição mais original do artigo – é um achado empírico, replicável, documentado por commit. Bem descrito aqui.

FALTOU DETALHE: Falta o número que comprova o achado: gráfico ou tabela de fitness por geração antes/depois da correção (commit a02d46a), nº de gerações, tamanho de população e fitness final do genoma embutido. Hoje o impacto é só descrito em texto, sem evidência quantitativa.

2.3 Projeto e implementação

2.3.1 Módulo de detecção de alvos

O módulo de detecção utiliza dois sensores ultrassônicos HC-SR04 instalados de costas um para o outro no topo de um mastro, na torre central. O mastro é movido por um motor de passo 28BYJ-48 que oscila 180°, de modo que, em conjunto, os dois sensores cobrem uma varredura de 360°. Cada sensor emite pulsos de 40 kHz e calcula a distância pelo atraso do eco, segundo a relação $d = \frac{v \cdot \Delta t}{2}$, em que v é a velocidade do som no ar (cerca de 343 m/s à temperatura ambiente). A cada leitura, o Arduino envia à aplicação desktop, pela porta serial, o ângulo e a distância em protocolo de texto simples, que a aplicação converte em posição cartesiana e exibe no radar circular de quatro quadrantes.

O protocolo é textual em ASCII, transportado pela porta serial USB a 9600 bauds por padrão, com cada mensagem terminada por `\n` e composta por um tipo em maiúsculas seguido de pares chave-valor separados por ponto e vírgula. Uma leitura de alvo chega na forma `TARGET;ID=1;ANGLE=45;DIST=2.80`, em que `ANGLE` é dado em graus no intervalo $[0, 360)$ e `DIST` em metros. O ângulo segue a convenção de bússola – 0° apontando para o Norte, sentido horário –, escolhida por ser a leitura mais intuitiva para quem opera um radar, e a conversão para coordenadas cartesianas de mundo, feita por uma rotina dedicada de conversão de coordenadas polares para cartesianas, é dada por $x = d \cdot \sin\theta$ e $y = d \cdot \cos\theta$, com θ em radianos. A partir do par (x, y), o quadrante do alvo é determinado pelos sinais das duas coordenadas (por um algoritmo auxiliar de identificação de quadrante) e a distância até cada torre candidata pela distância euclidiana clássica (calculada por uma rotina dedicada de cálculo de distância). Leituras com campos ausentes, não numéricos, com ângulo fora do intervalo válido ou com distância negativa são convertidas em mensagens de erro registradas no console de eventos, nunca em exceções – falhas de sensor ou de cabo produzem degradação graciosa, e não interrupção da aplicação.

2.3.2 Projeto elétrico e estrutural

O projeto elétrico é baseado em um Arduino Uno, plataforma disponível no simulador Tinkercad usado para simular o circuito. Os sinais dos motores de passo são gerados por registradores de deslocamento 74HC595 em cascata, e os motores 28BYJ-48 são acionados por drivers ULN2003, alimentados por uma fonte de 5 V/15 A. Cada uma das quatro torres armadas possui dois motores 28BYJ-48 (movimentos de pan e tilt), um laser de baixa potência e um LED de cor própria; o acionamento é exclusivamente demonstrativo. Na torre central, o mastro dos sensores não é acoplado diretamente ao eixo do motor: ele assenta em um disco que gira sobre um rolamento axial 51108, acionado pelo motor por um cubo duplo-D. A passagem dos fios fica próxima ao eixo central, com folga de cabo em vez de anel coletor (*slip ring*), já que o giro é limitado a 180°. A torre central é cerca de 5 cm mais alta que as torres armadas (cerca de 137,5 mm de altura). As peças estruturais são impressas em 3D, em formato de torre, com suporte para rolamento axial na base – o que permite a rotação da estrutura –, comum às cinco torres. As torres armadas têm, além disso, suportes próprios para os motores e para as câmeras, que cumprem o papel dos "canhões" de acionamento demonstrativo; a torre central tem suporte próprio para os sensores ultrassônicos no topo do mastro. Todas essas peças, assim como o restante do projeto, existem até o momento apenas em projeto digital (modelagem 3D e simulação elétrica), sem montagem física realizada.

2.3.3 Módulo de visão

O módulo de visão utiliza quatro câmeras de 720p conectadas por USB ao computador, associadas aos quadrantes do radar, e processa os quadros de vídeo (*frames*) para classificar o alvo por sua cor, formato e tamanho. Cada quadro capturado é entregue já decodificado à detecção, que executa localmente um modelo YOLO26, na variante nano, em formato ONNX, para propor as regiões candidatas da imagem. Cada candidata é então comparada ao perfil do alvo previamente cadastrado – definido por cor, formato e tamanho, e opcionalmente por fotos de referência recortadas no objeto de interesse.

Havendo foto de referência, a comparação combina um histograma de matiz e saturação com uma grade espacial de cor média, usados como assinatura do objeto; na ausência de foto, o critério recai sobre a fração de pixels com a cor cadastrada e sobre a classificação de formato. Quando o detector não propõe nenhuma caixa com a cor do perfil, as regiões candidatas passam a ser propostas por segmentação de cor, como último recurso na ordenação de candidatos.

O envio das informações resultantes ao Arduino, para guiar o apontamento da torre (movimentos de pan e tilt), foi projetado mas ainda não está implementado: o protocolo serial atual não possui comando de apontamento, e a fusão entre a posição estimada pela visão e a distância medida pelos sensores ultrassônicos permanece como etapa planejada. Como o hardware ainda não foi montado, a validação funcional descrita neste trabalho foi feita por simulação, e nenhum resultado de campo é relatado.

2.3.4 Fluxo operacional

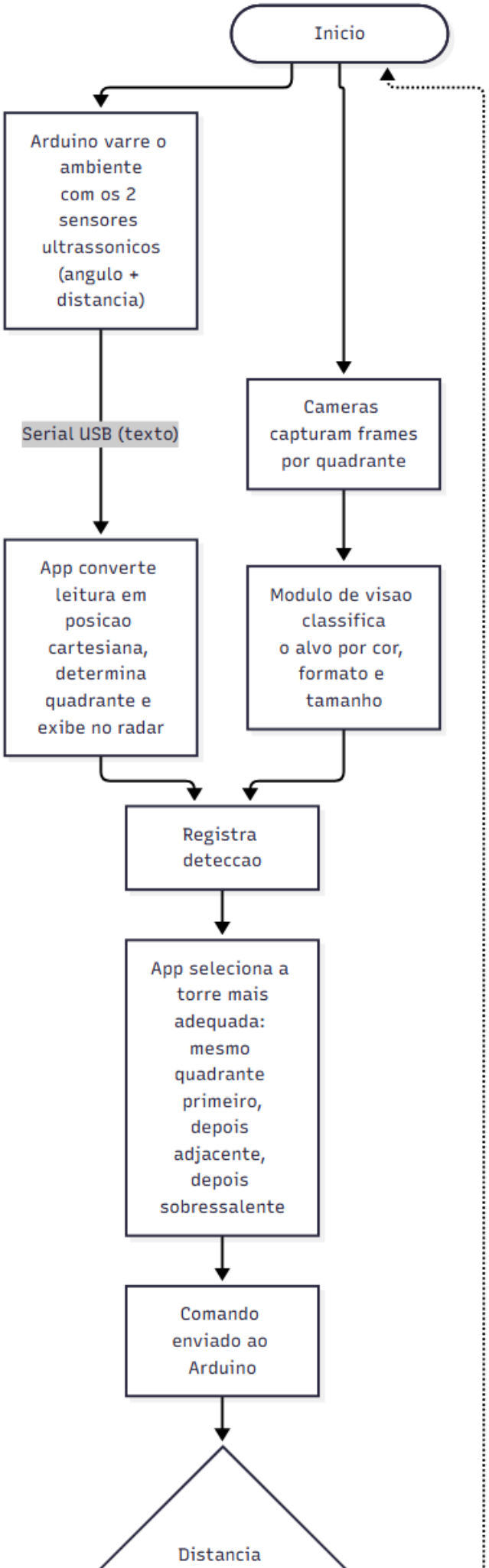
Em operação, o sistema segue estas etapas: (1) o Arduino varre o ambiente com os dois sensores ultrassônicos e envia ângulo e distância à aplicação; (2) a aplicação converte cada leitura em posição cartesiana, determina o quadrante, exibe o alvo no radar e registra a detecção; (3) em paralelo, as câmeras fornecem quadros de vídeo, e o módulo de visão classifica o alvo por cor, formato e tamanho; (4) a aplicação seleciona, entre as torres armadas disponíveis, a mais adequada para o alvo, priorizando as do mesmo quadrante e, entre elas, a de menor distância euclidiana (por proximidade, alvo a alvo); (5) o comando é enviado ao Arduino, que aciona a torre selecionada; (6) o acionamento demonstrativo (laser de baixa potência ou LED) só ocorre respeitando a distância mínima de segurança e as demais regras de segurança configuráveis do sistema; (7) toda ação executada é registrada em auditoria.

A seleção de torre e o acionamento dependem do modo de operação escolhido pelo operador, em um modelo de três níveis: no primeiro, os alvos são apenas detectados e exibidos; no segundo, acrescenta-se a seleção automática da torre, sem acionamento; no terceiro, acrescenta-se o acionamento demonstrativo automático. Quando a regra de segurança reprova um acionamento, ele é bloqueado e o motivo é registrado no console de eventos.

FICOU BOM: Os 7 passos do fluxo operacional estão bem numerados e em ordem lógica – fácil de seguir mesmo sem conhecimento técnico prévio. Fluxograma acima gerado a partir do mesmo texto, reforça visualmente o caminho completo do sinal, do sensor até o acionamento.

2.3.5 Controlador neuroevolutivo: heurística e rede neural combinadas

O passo (4) do fluxo operacional – escolher, entre as torres armadas disponíveis, a mais adequada para cada alvo, e decidir o instante do acionamento – é resolvido no RadarTorres pela combinação de dois mecanismos de natureza distinta: uma **heurística** determinística, escrita como regras explícitas de engenharia, e uma **rede neural** *feedforward*, cujo comportamento é aprendido por um algoritmo evolutivo próprio, sem biblioteca de aprendizado de máquina. Ambos coexistem no mesmo controlador híbrido, e é essa combinação – não cada mecanismo isoladamente – que determina tanto o comportamento em


Inicio
Arduino varre o ambiente com os 2 sensores ultrassonicos (angulo + distancia)
Serial USB (texto)
Cameras capturam frames por quadrante
App converte leitura em posicao cartesiana, determina quadrante e exibe no radar
Modulo de visao classifica o alvo por cor, formato e tamanho
Registra deteccao
App seleciona a torre mais adequada: mesmo quadrante primeiro, depois adjacente, depois sobressalente
Comando enviado ao Arduino
Distancia

produção quanto o que é efetivamente treinado.

A heurística é um conjunto de regras fixas, escritas e revisáveis por engenharia, cujo resultado para uma mesma entrada é sempre o mesmo. No RadarTorres ela resolve, em cascata, três camadas de prioridade na seleção da torre: primeiro considera apenas as torres livres do próprio quadrante do alvo; na ausência de candidata, estende-se às torres de quadrante adjacente cujo quadrante de origem não tenha outro alvo concorrente; por fim, na ausência de qualquer uma das duas, recorre a qualquer torre livre restante, de modo que nenhum alvo fique sem cobertura. Cada camada carrega um fator numérico (1,0; 0,5; ou 0,0) que soma-se diretamente à pontuação da candidata. A heurística também impõe cinco regras de segurança e refinamento, cada uma ativável individualmente no painel de Configurações: anti-oscilação, que evita trocar de torre por uma diferença de pontuação dentro de uma margem de empate; priorização por urgência, que permite a um alvo muito próximo da base "tomar" a torre de um alvo mais distante; corredor de tiro, que descarta candidatas cuja linha de disparo atravesse uma zona morta ativa; balanceamento de carga, que prefere, entre torres empatadas, a usada com menor frequência na sessão; e predição de trajetória, que projeta a posição futura do alvo para calcular o ângulo de mira com antecedência de poucos décimos de segundo. Por fim, a heurística impõe dois limites absolutos ao disparo, nunca delegados à rede: um tempo mínimo de mira de 0,3 s e um limiar mínimo de certeza de identificação do alvo (`LimiarConfiancaDisparoPercentual`), abaixo do qual o disparo é recusado independentemente de qualquer outra avaliação.

A rede neural, por sua vez, não decide essas regras de segurança nem a cascata de camadas – ela decide apenas a escolha fina *dentro* do espaço já validado pela heurística. É uma rede *feedforward* totalmente conectada, implementada do zero (sem biblioteca de aprendizado de máquina), com arquitetura de 6 entradas, uma camada oculta de 8 neurônios e 2 saídas, com ativação tanh em todas as camadas. As seis entradas – todas normalizadas no intervalo $[0, 1]$ ou equivalente – são a distância normalizada ao alvo, dois indicadores binários de quadrante (mesmo quadrante e quadrante adjacente), o ângulo de mira normalizado, a velocidade normalizada do alvo e o próprio fator da camada heurística vigente, usado como sinal de contexto. As duas saídas são um ajuste de pontuação, somado à pontuação heurística da candidata, e um valor de confiança de disparo, cujo sinal positivo autoriza o acionamento. Os pesos e vieses da rede – 74 parâmetros, no total, para essa arquitetura – não são ajustados por retropropagação, mas evoluídos por um algoritmo genético: cada genoma é um vetor plano desses 74 números, a população evolui por seleção por torneio, *crossover* uniforme, mutação gaussiana e elitismo, e o genoma final é embutido na aplicação como um vetor de pesos fixo, sem qualquer etapa de treinamento em tempo de execução.

A combinação entre os dois mecanismos é, portanto, sequencial e hierárquica, nunca uma mistura indiferenciada: a heurística primeiro filtra quais torres são elegíveis e calcula as *features* de entrada (quadrante, ângulo, fator de camada); a rede neural, em seguida, pondera essa pontuação dentro do subconjunto já validado, decidindo qual candidata específica vence o empate fino que a heurística, por si só, não resolveria com a mesma sensibilidade; e, por fim, a heurística volta a atuar como filtro de segurança sobre a decisão de disparo, vetando qualquer autorização da rede que viole o tempo mínimo de mira, a certeza mínima ou uma zona morta ativa. Essa hierarquia tem uma consequência deliberada para o pior caso: uma generalização imperfeita da rede nunca produz uma decisão arbitrária, porque toda saída da rede continua restrita ao espaço de candidatas e

ao conjunto de disparos já pré-aprovados pela heurística – no limite, o sistema regride ao comportamento puramente heurístico, nunca a um erro fora de controle.

Esse acoplamento se reflete diretamente no treinamento do controlador. O algoritmo genético não avalia a rede neural isolada: avalia o comportamento do `ControladorHibrido` completo – heurística e rede juntas – em episódios simulados de 45 s, com as torres reais do projeto, alvos nascendo na borda de um mundo circular e avançando em direção à base. O fitness de cada genoma soma recompensas por abate (maiores quanto mais longe da base o alvo é interceptado) e penalidades por alvo que escapa ou cruza um ponto de perda sem decisão, de modo que o genoma é pressionado a aprender a operar bem *dentro* das restrições que a heurística já impõe, e não a contorná-las – não há genoma "vencedor" que dispare fora da zona morta ou abaixo da certeza mínima, porque essas decisões nunca chegam a ser avaliadas como tal pelo ambiente de treino. Essa mesma dependência, no entanto, também é o ponto mais sensível do processo: como o fitness depende inteiramente do comportamento simulado, qualquer premissa equivocada no ambiente de treino se propaga, silenciosamente, para o genoma evoluído. Foi exatamente esse o caso identificado e corrigido neste trabalho – a certeza de identificação do alvo era sorteada uma única vez, no nascimento, e nunca era atualizada, de modo que cerca de 70% dos alvos simulados nasciam permanentemente abaixo do limiar mínimo de disparo e jamais podiam ser abatidos, independentemente da qualidade do genoma, o que mascarava qualquer evolução real sob a aparência de estagnação. A correção – fazer a certeza crescer com o tempo de observação do alvo, como ocorreria com uma câmera real acumulando quadros – e o alinhamento posterior dos parâmetros de alcance de mira e de limiar de certeza usados no treino com os mesmos valores configuráveis em produção (`ConfiguracoesAplicativo.AlcanceMiraMetros`, `LimiarConfiancaDisparoPercentual`) ilustram, em conjunto, por que a fidelidade do ambiente simulado ao comportamento do sistema real é, para um controlador neuroevolutivo, tão determinante para o resultado do treinamento quanto a própria arquitetura da rede ou os hiperparâmetros da busca genética.

FICOU BOM: Explicar a hierarquia heurística → rede → heurística (filtra, pondera, veta) ficou claro e tecnicamente preciso – é o núcleo mais original do artigo.

FALTOU DETALHE: Falta um diagrama simples desse fluxo (heurística filtra candidatas → rede decide dentro do filtro → heurística veta disparo inseguro) – hoje só em texto, e é uma ideia que se beneficia bastante de uma figura, inclusive pra reforçar visualmente que a mesma classe `ControladorHibrido` é usada em treino e produção.

2.4 Diagrama de pacotes

A organização interna da aplicação desktop foi documentada por meio de um diagrama de pacotes, diagrama estrutural da UML que representa a divisão lógica do sistema em pacotes e as dependências existentes entre eles, sem detalhar atributos ou métodos das classes neles contidas. No RadarTorres, cada pacote corresponde a um *namespace* do projeto `RadarTorres.App`, coincidente com uma pasta do código-fonte, e o diagrama foi obtido diretamente do código da *branch* principal do repositório: cada seta representa ao menos uma diretiva `using RadarTorres.App.*` (ou uma referência `clr-namespace` em XAML) presente nos arquivos do pacote de origem, de modo que o diagrama descreve a

estrutura efetivamente implementada, e não apenas a pretendida.

O fluxo principal de dependências segue a organização do padrão MVVM, de cima para baixo: Views → ViewModels → Services → Repositories → Data → Models. O pacote Models ocupa a posição mais estável do sistema – todos os demais dependem dele e ele não depende de nenhum outro –, o que é desejável para as entidades de domínio, e a separação entre as interfaces de Repositories e a implementação em CSV do pacote Data é o que viabiliza a futura migração para um banco relacional sem alterar ViewModels ou Services. A construção do diagrama a partir do código revelou, também, três dependências que formam ciclos entre pacotes – destacadas em vermelho na figura –: o `NavigationService` importa *namespaces* de Views para instanciar telas (Services → Views), e o `DataSeeder`, responsável pela carga inicial de dados, depende de Repositories e de Services, enquanto esses mesmos pacotes dependem de Data. Esses ciclos não impedem o funcionamento da aplicação, pois todos os pacotes pertencem ao mesmo *assembly*, mas aumentam o acoplamento e dificultam testes isolados e futuras separações em projetos distintos.

FICOU BOM: Diagrama regenerado a partir do código atual – já inclui o pacote `NeuralEvolution` e as dependências novas (`Services` e `ViewModels` o importam).

FALTOU DETALHE: A versão anterior da figura destacava em vermelho os três ciclos de dependência encontrados (`NavigationService`, `DataSeeder`). A regeração simplificada acima não reproduz esse destaque – se os ciclos ainda existirem no código atual, vale redesenhar com a mesma marcação, em vez de simplesmente perder essa informação.

2.5 Modelo de dados

MUDAR: Achado grave: a persistência não é mais CSV – já migrou pra **SQLite com chaves estrangeiras reais** (`SqliteSchema.cs` e os `Sqlite*Repository.cs`). O texto e a figura DER_Atual.png descrevem um estágio anterior do projeto. Tabelas reais hoje: 11, não 6 – `Usuarios`, `ObjetosDetectados`, `OcorrenciaEstados` (nova), `acoes_realizadas`, `modo_atual_torre`, `PreferenciasUsuario`, `Cameras` (nova), `classificacoes_alvo` (nova), `historico_treinamento_ geracoes` (nova), `marcos_fitness_treinamento` (nova), `instantaneos_comportamento_melhor_gene` (nova). `ChamadoAjuda`, a 6ª entidade do artigo atual, não existe mais no código. `PreferenciasUsuario.UsuarioId` e `modo_atual_torre.ID_USER` já são FK reais por inteiro (com `PRAGMA foreign_keys = ON`), não mais vínculo por login/texto como o artigo afirma. Já existe documentação mais atualizada em `Docs/Documentos_Entregaveis/Banco_de_Dados/conteudo_modelo_banco_dados.tex` e uma figura `DER_SQLite.png` prontas pra reaproveitar – mas mesmo esse documento ainda não inclui as 4 tabelas de treinamento neuroevolutivo listadas acima. Atualizar o artigo a partir desses dois documentos, cobrindo as 11 tabelas reais.

A persistência do RadarTorres foi igualmente documentada por meio de um levantamento do modelo de dados diretamente a partir do código-fonte já implementado. O protótipo não utiliza, no estágio atual, um sistema gerenciador de banco de dados relacional: seis entidades – usuários, objetos detectados, ações realizadas, trocas de modo de operação, preferências de usuário e chamados de suporte – são persistidas localmente em arquivos CSV, um por tabela, cada uma acessada exclusivamente por uma interface de repositório dedicada, o que isola as camadas de ViewModel e Service do formato de armazenamento.

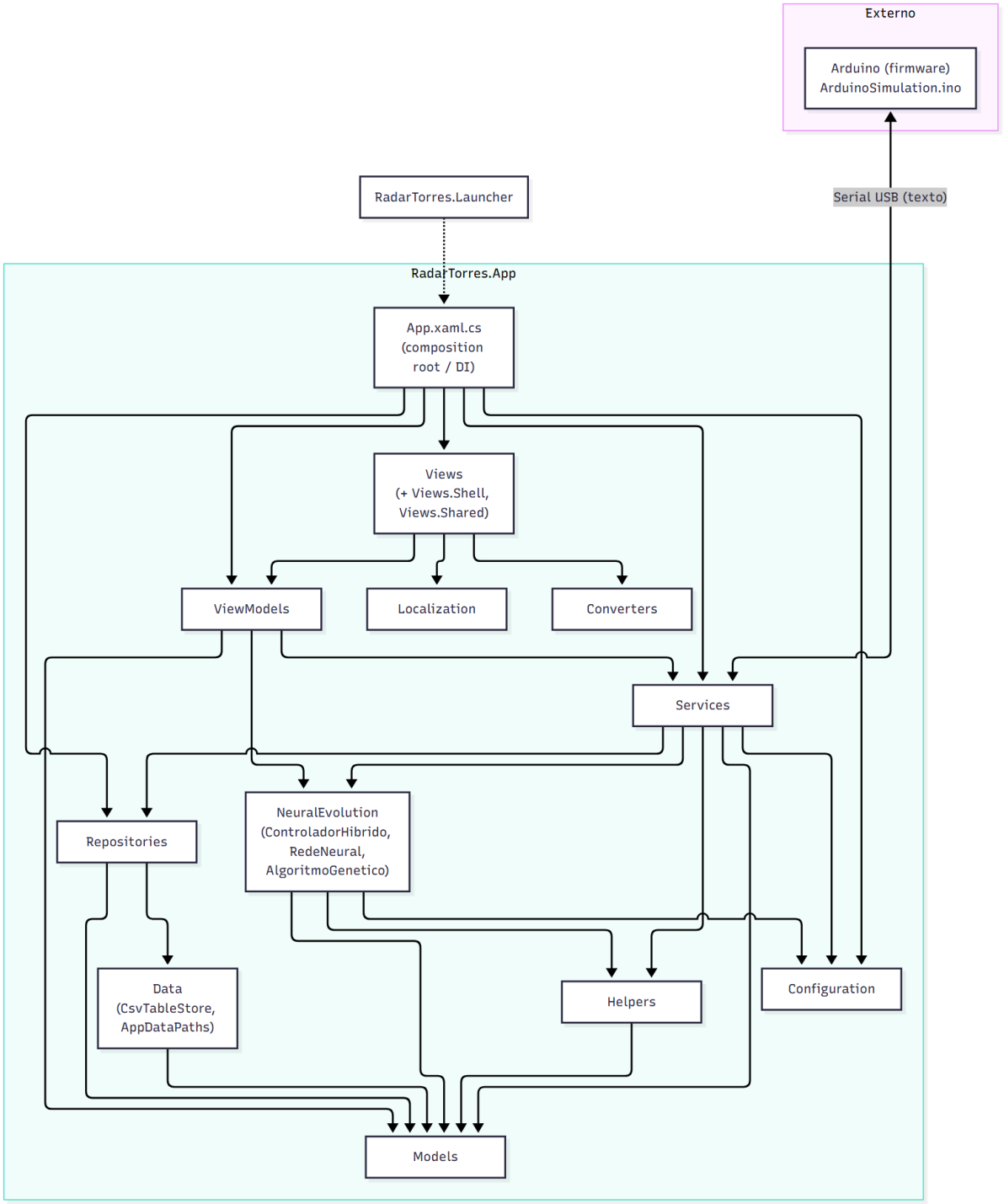


Figure 2: Diagrama de pacotes do RadarTorres. Setas em vermelho indicam dependências que formam ciclos (dos autores).

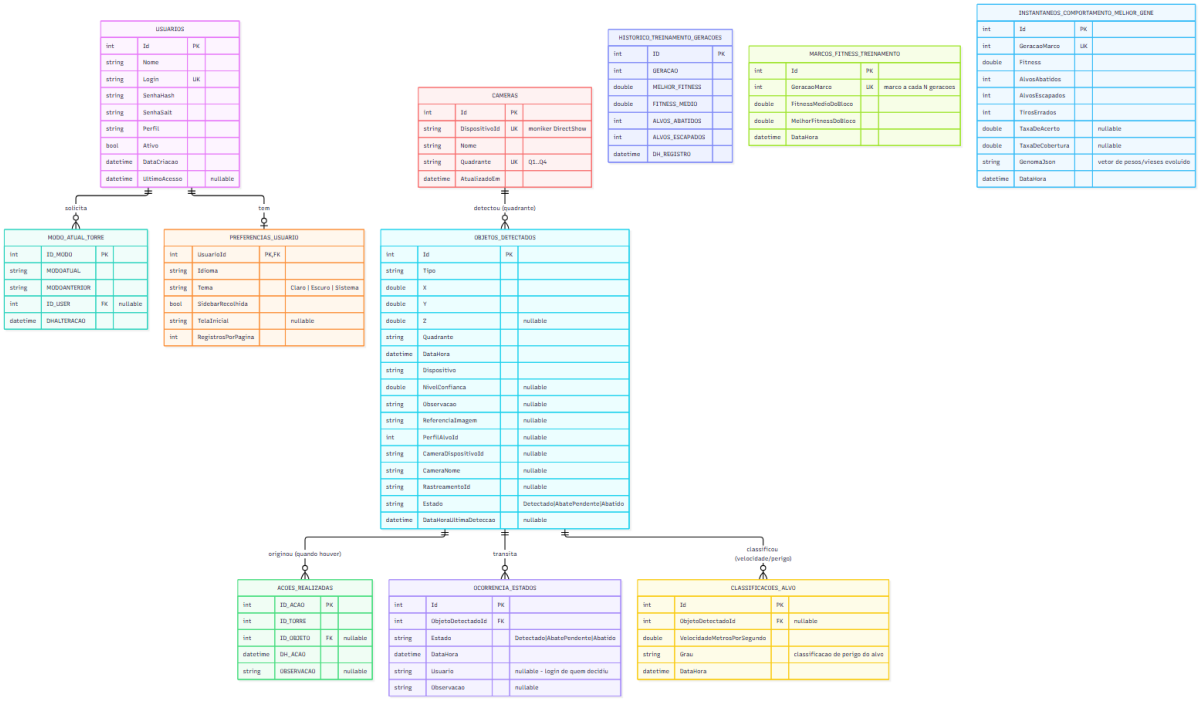


Figure 3: Modelo de dados atual (SQLite, 11 tabelas, chaves estrangeiras reais) – diagrama entidade-relacionamento levantado a partir do código-fonte do projeto (dos autores).

FALTOU DETALHE: Esta figura (nova, SQLite) e a figura abaixo (DER_Atual.png, modelo antigo em CSV) ainda coexistem no artigo. O texto corrido acima e abaixo da figura antiga ainda descreve o modelo em CSV sem chave estrangeira real – precisa ser reescrito para as 11 tabelas e as FKs reais já existentes (ver comentário anterior). A figura DER_Atual.png pode virar nota histórica ("modelo original, já superado pela migração para SQLite") em vez de ser apresentada como estado atual.

`Usuario` se relaciona 1:1 com `PreferenciasUsuario` e 1:N com `AcaoRealizada` e `ModoAtualTorre` – nesses dois últimos casos por **login** (texto), sem chave estrangeira real – e com `ChamadoAjuda`, por identificador numérico. `ObjetoDetectado` não possui nenhum vínculo persistido com as demais entidades, sendo registrado de forma independente. Como proposta de evolução, foi elaborado um modelo relacional equivalente, no qual as referências textuais por login seriam substituídas por chaves estrangeiras íntegras por identificador numérico, com integridade garantida pelo próprio SGBD, permitindo migrar a persistência de CSV/JSON para SQL sem exigir alterações nas camadas de ViewModel ou Service, já isoladas do formato de armazenamento por meio das interfaces de repositório.

MUDAR: Falta uma seção de Conclusão/Considerações finais entre "Desenvolvimento" e as referências: hoje o artigo termina abruptamente no modelo de dados, sem sintetizar resultados, consolidar limitações (hoje espalhadas em vários parágrafos) nem unificar os trabalhos futuros.

Referências

BAEK, I. et al. *CurbScan: Curb Detection and Tracking Using Multi-Sensor Fusion.* 2020.

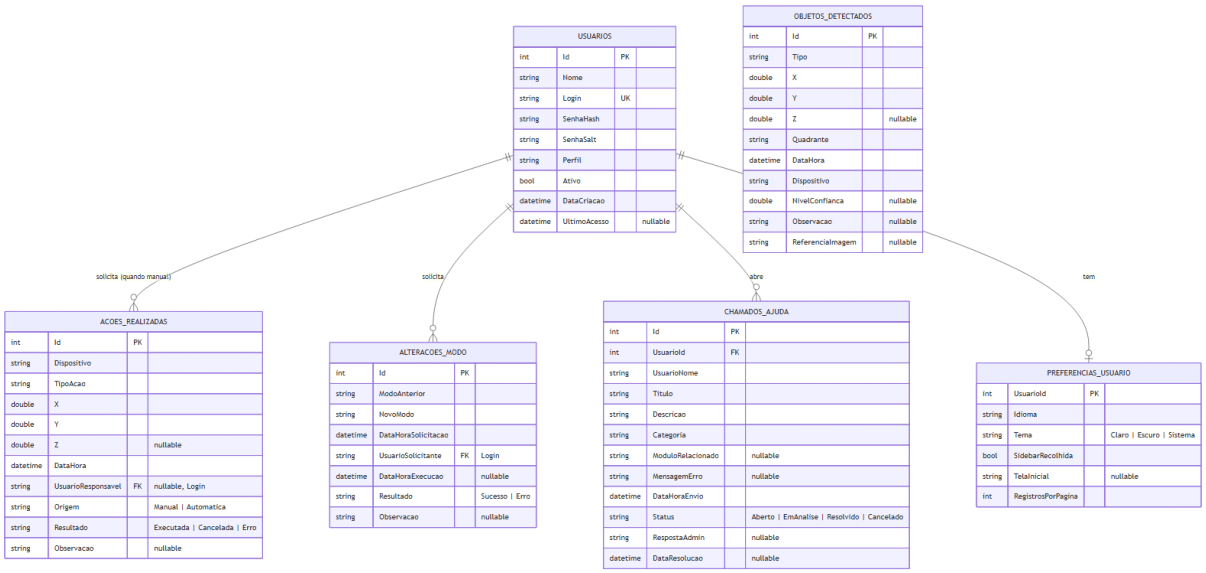


Figure 4: Modelo de dados original em CSV, já substituído pela migração para SQLite (ver figura anterior) – mantido aqui como registro histórico do projeto (dos autores).

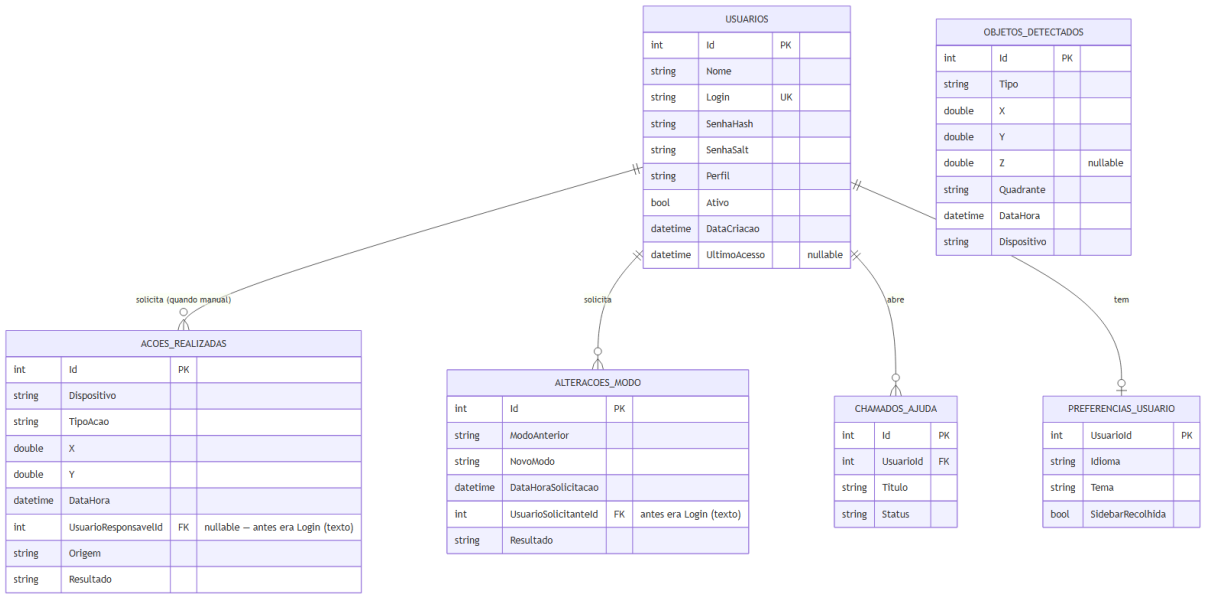


Figure 5: Modelo relacional proposto para a migração futura de persistência, de CSV/JSON para SQL (dos autores).

MAJONI, L. et al. Camera and ultrasonic sensor fusion for electronic travel aid: design and performance analysis. *MATEC Web of Conferences*, v. 417, p. 10005, 2025.

SUCH, F. P. et al. *Deep Neuroevolution: Genetic Algorithms Are a Competitive Alternative for Training Deep Neural Networks for Reinforcement Learning*. 2017.